



Week 4 - File Handling, Static and Dynamic Typing

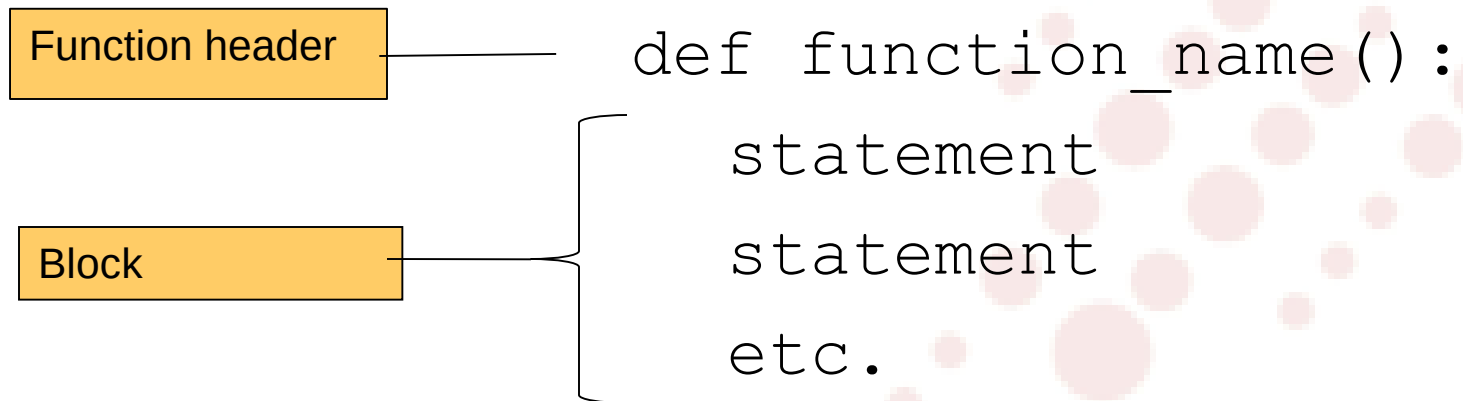


What is in this lesson?

1. Defining and Calling a Function(Recap)
2. **Value-Returning Functions and Modules**
 - Standard Library Functions
 - Write Value-Returning Functions
3. File Handling
 - File Input and Output
 - Using Loops to Process Files
 - Processing Files -> Records/Objects
4. Exceptions

1. Defining and Calling a Function(Recap)

Defining and Calling a Function



1. Defining and Calling a Function(Recap)

Defining and Calling a Function

The function definition and the function call

These statements cause
the `message` function to
be created.

```
# This program demonstrates a function.  
# First, we define a function named message.  
def message():  
    print 'I am Arthur,'  
    print 'King of the Britons.'
```

```
# Call the message function.  
message()
```

This statement calls
the `message` function,
causing it to execute.

2. Value-Returning Functions

Value-returning function:

- **A group of statements that performs a specific task**
- **Executed when called**
- **Returns a value back to the part of the program that called it**

2.1 Standard Library Functions

Standard Library Functions: prewritten functions that perform commonly needed tasks.

- Already written and come with the language
- Make a programmer's job easier
- Perform many commonly needed tasks
- Stored in files known as *modules*
- Include the `import` statement at the top of the program



For example:

`input()` `raw_input()` `range()`

2.1. Standard Library Functions

Generating Random Numbers

For example:

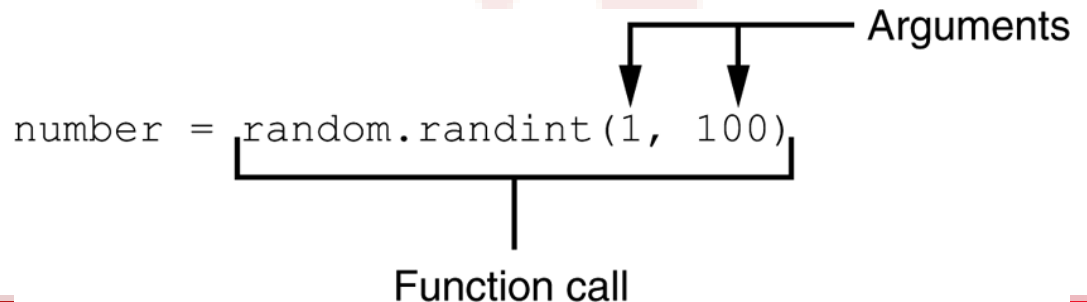
causes the interpreter to load the contents of the random module

```
import random
```

causes the interpreter to load the contents of

```
number = random.randint(1, 100)
```

Figure 6-2 A statement that calls the `random` function



2.1 Standard Library Functions

Generating Random Numbers

Program (random_numbers2.py)

```
1  # This program displays five random
2  # numbers in the range of 1 through 100.
3  import random
4
5  def main():
6      for count in range(5):
7          # Get a random number.
8          number = random.randint(1, 100)
9          # Display the number.
10         print number
11
12     # Call the main function.
13     main()
```


2.1. The math Module

Program (square_root.py)

```
1  # This program demonstrates the sqrt function.
2  import math
3
4  def main():
5      # Get a number.
6      number = input('Enter a number: ')
7
8      # Get the square root of the number.
9      square_root = math.sqrt(number)
10
11     # Display the square root.
12     print 'The square root of', number, 'is', square_root
13
14     # Call the main function.
15     main()
```

2.2. Writing Your Own Value-Returning Functions

Concept:

A value-returning function has a return statement that returns value back to the part of the program that called it.

2.2 Writing Your Own Value-Returning Functions

Using IPO Charts

- A tool used for designing and documenting functions
- IPO stands for *Input*, *Processing*, and *Output*
- Describes the input, processing, and output of a function

IPO Chart for the <code>get_regular_price</code> Function		
Input	Processing	Output
None	Prompts the user to enter an item's regular price	The item's regular price

IPO Chart for the <code>discount</code> Function		
Input	Processing	Output
An item's regular price	Calculates an item's discount by multiplying the regular price by the global constant <code>DISCOUNT_PERCENTAGE</code> .	The item's discount

2.2 Writing Your Own Value-Returning Functions

```
def
function_name() :
    statement
    statement
    etc.
    return
expression
```

Figure 6-5 Parts
of the function

The diagram shows a function definition with arrows pointing to its components:

```
def sum(num1, num2):
    result = num1 + num2
    return result
```

Annotations:

- "The name of this function is sum." points to `sum`.
- "num1 and num2 are parameters." points to `num1, num2`.
- "This function returns the value referenced by the result variable." points to `result` in the `return` statement.

2.2 Writing Your Own Value-Returning Functions

Program (total_ages.py)

```
1  # This program uses the return value of a function.
2
3  def main():
4      # Get the user's age.
5      first_age = input('Enter your age: ')
6
7      # Get the user's best friend's age.
8      second_age = input("Enter your best friend's age: ")
9
10     # Get the sum of both ages.
11     total = sum(first_age, second_age)
12
13     # Display the total age.
14     print 'Together you are', total, 'years old.'
15
16     # The sum function accepts two numeric arguments and
17     # returns the sum of those arguments.
18     def sum(num1, num2):
19         result = num1 + num2
20         return result
21
22     # Call the main function.
23     main()
```

2.3 Writing Your Own Value-Returning Functions

Making the Most of the `return` statement

```
def sum(num1, num2):  
    result = num1 + num2  
    return result
```

```
def sum(num1, num2):  
    return num1 + num2
```

2.3 Writing Your Own Value-Returning Functions

Returning Multiple Values

```
first_name, last_name = get_name()
```

```
def get_name():  
    # Get the user's first and last name.  
    first = raw_input('Enter your first name: ')  
    last = raw_input('Enter your last name: ')  
  
    # Return both names.  
    return first, last
```

2.3 Storing Functions In Modules

```
def read_string(prompt):  
    return input(prompt)  
  
def read_float(prompt):  
    return float(read_string(prompt))  
  
def read_integer(prompt):  
    return int(read_string(prompt))  
  
def read_integer_in_range(prompt,  
    min_value, max_value):  
    value = read_integer(prompt)  
    while value < min_value or value >  
max_value:  
        print(f"Please enter a value  
between {min_value} and {max_value}.")  
        value = read_integer(prompt)  
    return value
```

```
def read_boolean(prompt):  
    value = read_string(prompt).lower()  
    return value in ['y', 'yes']  
  
def print_float(value, decimal_places):  
    print(round(value, decimal_places))
```

Storing Functions In Modules

- Module file names should end in .py (input_functions.py)
- Place the module in the same folder as the program
- Using: **import input_functions**

File Handling

3.1 Introduction to File Input and Output

Concept:

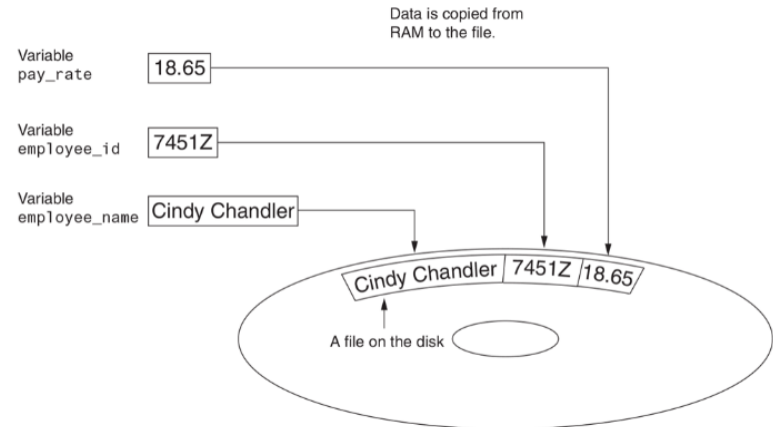
When a program needs to save data for later use, it writes the data in a file. The data can be read from the file at a later time.

3.1 Introduction to File Input and Output

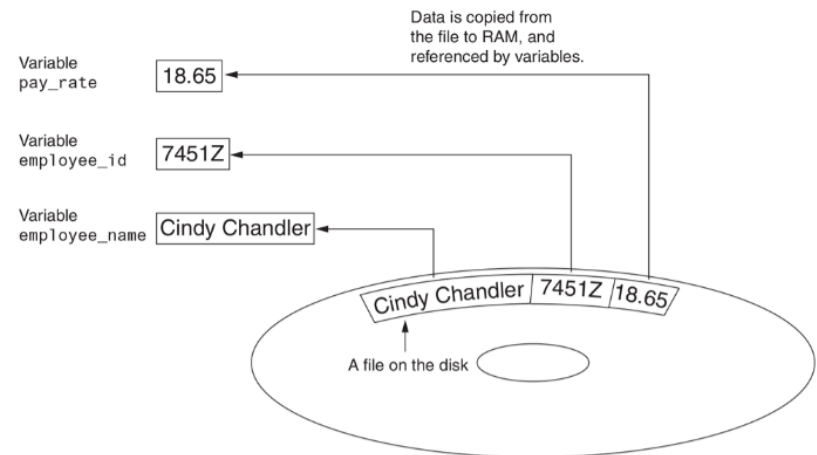
Terms

- Saving data in a file = “writing data to” the file
- **Output file** = file that data is written to
- Retrieving data from a file = “reading data from” the file
- **Input file** = file that data is read from

Writing data to a file



Reading data from a file



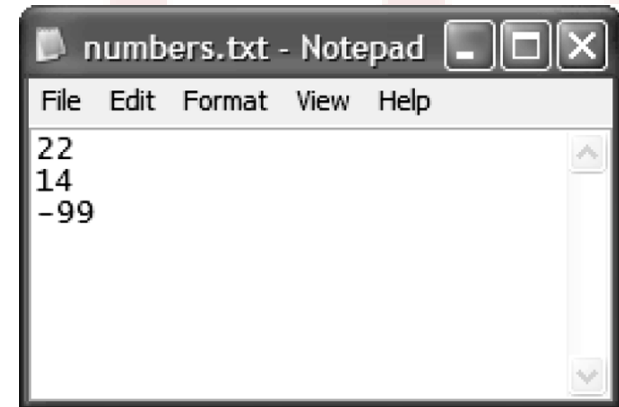
3.1 Introduction to File Input and Output

Types of Files

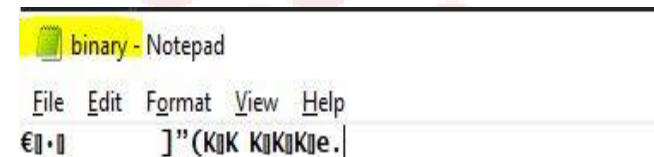
Two types of files:

- **Text file** - contains data that has been encoded as text, ASCII or Unicode
- **Binary file** - contains data that has not been converted to text

Text file



Binary file



3.1 Introduction to File Input and Output

File Access Methods

Two ways to access data stored in files:

- ***Sequential Access*** - access data from the beginning of the file to the end of the file
- ***Direct (random) Access*** - directly access any piece of data in the file without reading the data that comes before it

3.1 Introduction to File Input and Output

Opening a File

The `open` function:

- General format: `file_variable = open(filename, mode)`
- *file_variable* is the name of the variable that will reference the file object
- *filename* is a string specifying the name of the file
- *mode* is a string specifying the mode (reading, writing, etc.)

Mode	Description
'r'	Open a file for reading only. The file cannot be changed or written to.
'w'	Open a file for writing. If the file already exists, erase its contents. If it does not exist, create it.
'a'	Open a file to be written to. All data written to the file will be appended to its end. If the file does not exist, create it.

```
customer_file = open('customers.txt', 'r')
```

```
sales_file = open('sales.txt', 'w')
```

3.1 Introduction to File Input and Output

Writing Data to a File

Method - function that belongs to an object

- perform operation using that object

file_variable.write(string)

- **file_variable** - variable that references a file object
- **write** - file object used to write data to a file
- **string** - string that will be written to the file

```
1  # This program writes three lines of data
2  # to a file.
3  def main():
4      # Open a file named philosophers.txt.
5      outfile = open('philosophers.txt', 'w')
6
7      # Write the names of three philosophers
8      # to the file.
9      outfile.write('John Locke\n')
10     outfile.write('David Hume\n')
11     outfile.write('Edmund Burke\n')
12
13     # Close the file.
14     outfile.close()
15
16 # Call the main function.
17 main()
```

3.1 Introduction to File Input and Output

Reading Data from a File ... read method

Program 7-2 (file_read.py)

```
1  # This program reads and displays the contents
2  # of the philosophers.txt file.
3  def main():
4      # Open a file named philosophers.txt.
5      infile = open('philosophers.txt', 'r')
6
7      # Read the file's contents.
8      file_contents = infile.read()
9
10     # Close the file.
11     infile.close()
12
13     # Print the data that was read into
14     # memory.
15     print file_contents
16
17 # Call the main function.
18 main()
```

```
1  # This program reads the contents of the
2  # philosophers.txt file one line at a time.
3  def main():
4      # Open a file named philosophers.txt.
5      infile = open('philosophers.txt', 'r')
6
7      # Read three lines from the file.
8      line1 = infile.readline()
9      line2 = infile.readline()
10     line3 = infile.readline()
11
12     # Close the file.
13     infile.close()
14
15     # Print the data that was read into
16     # memory.
17     print line1
18     print line2
19     print line3
20
21 # Call the main function.
22 main()
```


3.2 Using Loops to Process Files

Concept:

Files usually hold large amounts of data, and programs typically use a loop to process the data in a file.

3.2 Using Loops to Process Files

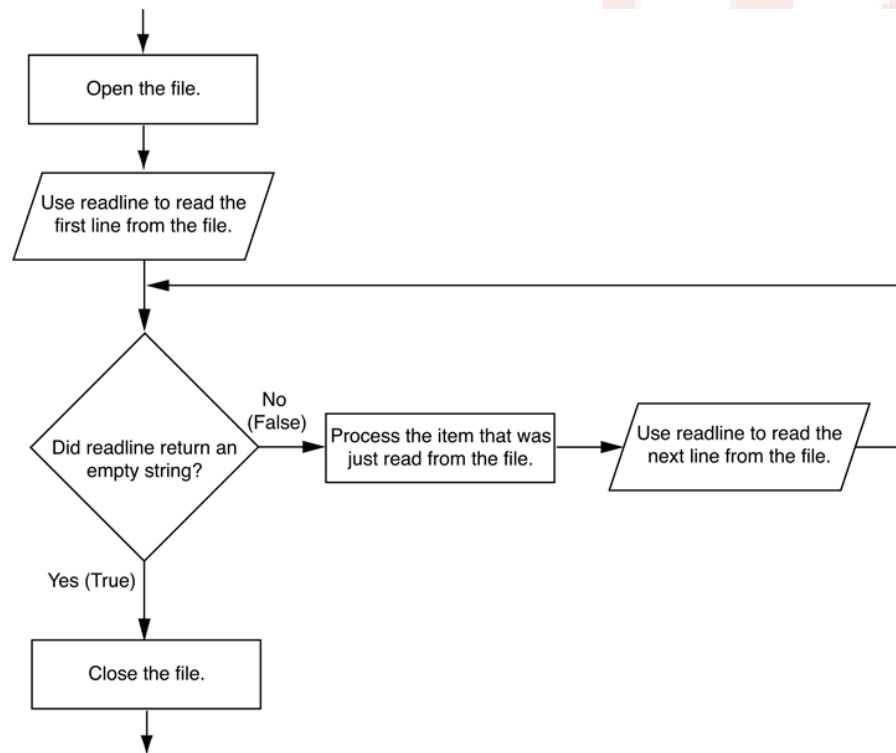
Reading a File with a Loop and Detecting the End of the File

- Read the contents of a file without knowing the number of items that are stored in the file
- `readline` method returns an empty string (‘ ‘) when it attempts to read beyond the end of file
- Priming `read` is required to test loop condition, if using a `while` loop

3.2 Using Loops to Process Files

Reading a File with a Loop and Detecting the End of the File

Figure 7-17 General logic for detecting the end of a file



3.2 Using Loops to Process Files

Using Python's `for` Loop to Read Lines

- `for` loop automatically reads a line of text from the input file
- No special condition or testing is needed
- No priming read is needed
- Automatically stops when the end of file is reached

```
for variable in file_object:  
    statement  
    statement  
    etc.
```

3.2 Using Loops to Process Files

Using Python's for Loop to Read Lines

Program 7-10 (read_sales2.py)

```
1  # This program uses the for loop to read
2  # all of the values in the sales.txt file.
3
4  def main():
5      # Open the sales.txt file for reading.
6      sales_file = open('sales.txt', 'r')
7
8      # Read all the lines from the file.
9      for line in sales_file:
10         # Convert line to a float.
11         amount = float(line)
12         # Format and display the amount.
13         print '$%.2f' % amount
14
15     # Close the file.
16     sales_file.close()
17
18 # Call the main function.
19 main()
```

3.3 Processing Records

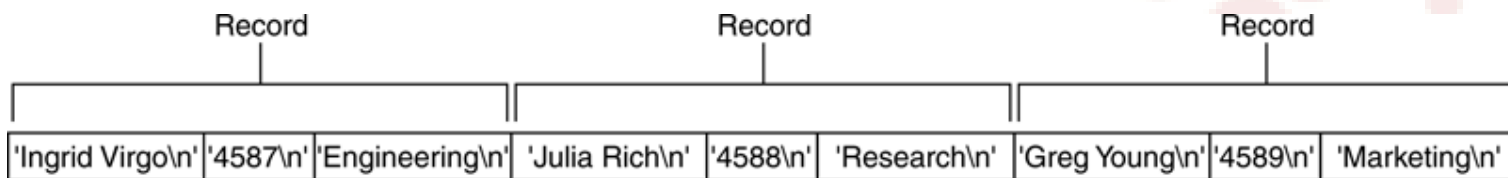
Concept:

The data that is stored in a file is frequently organized in records. A record is a complete set of data about an item, and a field is an individual piece of data within a record.

3.3 Processing Records

- A file's data is organized into records and fields
- **Record** – a complete set of data that describes one item
- **Field** – a single piece of data within a record

Figure 7-19 Records in a file



3.3 Processing Records

File manipulations

- **Create** an employee records file
- **Read** an employee records file

3.3 Processing Records

Creating an employee records file ... **employee.txt**

- 1. Get the total number of employees**
- 2. Open the file for writing**
- 3. For each employee**
 - a. Get the data for an employee**
 - b. Pad each field with newline character**
 - c. Write the employee record to the file**
- 4. Close the file**

3.3 Processing Records

Creating an employee records file ... employee.txt

Program 7-13 (save_emp_records.py)

```
1 # This program gets employee data from the user and
2 # saves it as records in the employee.txt file.
3
4 def main():
5     # Get the number of employee records to create.
6     num_emps = input('How many employee records ' + \
7                     'do you want to create? ')
8
9     # Open a file for writing.
10    emp_file = open('employees.txt', 'w')
11
12    # Get each employee's data and write it to
13    # the file.
14    for count in range(1, num_emps + 1):
15        # Get the data for an employee.
16        print 'Enter data for employee #' + str(count)
17        name = raw_input('Name: ')
18        id_num = raw_input('ID number: ')
19        dept = raw_input('Department: ')
20
21        # Write the data as a record to the file.
22        emp_file.write(name + '\n')
23        emp_file.write(id_num + '\n')
24        emp_file.write(dept + '\n')
25
26        # Display a blank line.
27        print
28
29        # Close the file.
30        emp_file.close()
31        print 'Employee records written to employees.txt.'
32
33    # Call the main function.
34    main()
```

3.3 Processing Records

Reading an employee records file ... `employee.txt`

- 1. Open the file for reading**
- 2. Read the first line from file**
- 3. While NOT end-of-file**
 - a. Read employee record**
 - b. Strip the newlines from the each field**
 - c. Display employee record**
- 4. Close the file**

3.3 Processing Records

**Reading an employee
records file ...
employee.txt**

Program 7-14
(read_emp_records.py)

```
1 # This program displays the records that are
2 # in the employees.txt file.
3
4 def main():
5     # Open the employees.txt file.
6     emp_file = open('employees.txt', 'r')
7
8     # Read the first line from the file, which is
9     # the name field of the first record.
10    name = emp_file.readline()
11
12    # If a field was read, continue processing.
13    while name != '':
14        # Read the ID number field.
15        id_num = emp_file.readline()
16
17        # Read the department field.
18        dept = emp_file.readline()
19
20        # Strip the newlines from the fields.
21        name = name.rstrip('\n')
22        id_num = id_num.rstrip('\n')
23        dept = dept.rstrip('\n')
24
25        # Display the record.
26        print 'Name:', name
27        print 'ID:', id_num
28        print 'Dept:', dept
29        print
30
31        # Read the name field of the next record.
32        name = emp_file.readline()
33
34    # Close the file.
35    emp_file.close()
36
37    # Call the main function.
38    main()
```

4 Exceptions

Concept:

An exception is an error that occurs while a program is running, causing the program to abruptly halt. You can use the `try/exception` statement to gracefully handle exceptions.

4 Exceptions

- Run time error
- Causes program to abruptly halt
- Error message is displayed ... *traceback* – information regarding the cause of the exception
- For example:

ZeroDivisionError - division by zero

IOError – file specified does not exist

4 Exceptions

ZeroDivisionError - division by zero

Program 7-20 (division.py)

```
1  # This program divides a number by another number.
2
3  def main():
4      # Get two numbers.
5      num1 = input('Enter a number: ')
6      num2 = input('Enter another number: ')
7
8      # Divide num1 by num2 and display the result.
9      result = num1 / num2
10     print num1, 'divided by', num2, 'is', result
11
12 # Call the main function.
13 main()
```

Program Output (with input shown in bold)

```
Enter a number: 10 [Enter]
Enter another number: 0 [Enter]
Traceback (most recent call last):
  File "C:/Python/division.py," line 13, in <module>
    main()
  File "C:/Python/division.py," line 9, in main
    result = num1 / num2
ZeroDivisionError: integer division or modulo by zero
```

4 Exceptions

IOError – file specified does not exist

Program 7-22 (display_file.py)

```
1 # This program displays the contents
2 # of a file.
3
4 def main():
5     # Get the name of a file.
6     filename = raw_input('Enter a filename: ')
7
8     # Open the file.
9     infile = open(filename, 'r')
10
11     # Read the file's contents.
12     contents = infile.read()
13
14     # Display the file's contents.
15     print contents
16
17     # Close the file.
18     infile.close()
19
20 # Call the main function.
21 main()
```

Program Output (with input shown in bold)

Enter a filename: **bad_file.txt** [Enter]

Traceback (most recent call last):

File "C:/Python/display_file.py," line 21, in <module>
main()

File "C:/Python/display_file.py," line 9, in main
infile = open(filename, 'r')

IOError: [Errno 2] No such file or directory: 'bad_file.txt'

4 Exceptions

- **Exception handler** prevents the program from abruptly crashing
- Use the `try/except` statement

```
try:  
    statement  
    statement  
    etc.
```

} try block

```
except ExceptionName:  
    statement  
    statement  
    etc.
```

except clause

} handler

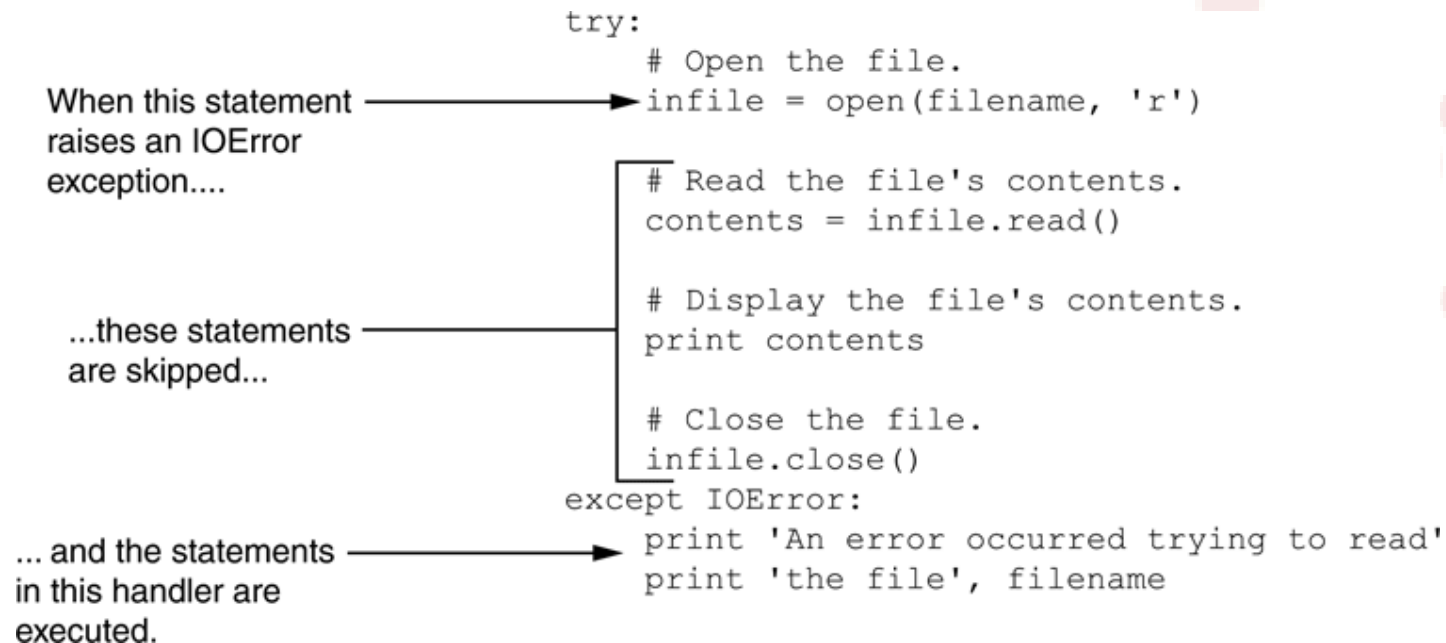
4. Exceptions

When `try/except` statement executes, the statements in the ***try block*** begin to execute:

- If a statement in the **try block** raises an exception that is specified by the ***ExceptionName*** in an **except clause**, then the **handler** that immediately follows the **except clause** executes. Then, the program resumes execution with the statement immediately following the `try/except` statement.
- If a statement in the **try block** raises an exception that is not specified by the ***ExceptionName*** in an **except clause**, then the program will halt with a traceback error message.
- If the statements in the **try block** execute without raising an exception, then any **except clauses** and **handlers** in the statement are skipped and the program resumes execution with the statements immediately following the `try/except` statement.

4 Exceptions

Sequence of events in the try/except statement



4 Exceptions

Program (sales_report.py)

```
1  # This program displays the total of the
2  # amounts in the sales_data.txt file.
3
4  def main():
5      # Initialize an accumulator.
6      total = 0.0
7
8      try:
9          # Open the sales_data.txt file.
10         infile = open('sales_data.txt', 'r')
11
12         # Read the values from the file and
13         # accumulate them.
14         for line in infile:
15             amount = float(line)
16             total += amount
17
18         # Close the file.
19         infile.close()
20
21         # Print the total.
22         print 'Total: $%.2f' % total
23
24     except IOError:
25         print 'An error occurred trying to read the file.'
26
27     except ValueError:
28         print 'Non-numeric data found in the file.'
29
30     except:
31         print 'An error occurred.'
32
33 # Call the main function.
34 main()
```

Review

1. Defining and Calling a Function(Recap)
2. **Value-Returning Functions and Modules**
 - Standard Library Functions
 - Write Value-Returning Functions
3. File Handling
 - File Input and Output
 - Using Loops to Process Files
 - Processing Files -> Records/Objects
4. Exceptions

References



Gaddis, Tony *Starting out with Python*. Pearson Education Limited. Sixth Edition, 2024.

Any Question?

